

Software Design Principles

SOLID & More

Software design principles represent a set of guidelines that helps us to avoid having a bad design



Abstract Design Guideline

Cohesion: Cohesion is the degree of togetherness between modules, classes, or attributes in a software system. It describes how closely related the responsibilities and elements inside a module or class are.

- Related code should be close to each other
- Bind all related code together as close as possible.

Coupling: Coupling is the degree of dependency between different modules or classes. It describes how much one module or class depends on another.

- Modules/Classes should be independent as far as possible. It helps to **reduce dependency** and make the **system easier to maintain**.



Art of Unix Programming

The Art of Unix Programming is a set of design principles and practices that describe how to create software in the Unix philosophy style.

- **Rule of Clarity:** Clarity is better than cleverness.
- **Rule of Composition:** Design programs to be connected to other programs.
- **Rule of Separation:** Separate policy from mechanism; separate interfaces from engines
- **Rule of Simplicity:** Design for simplicity; add complexity only where you must



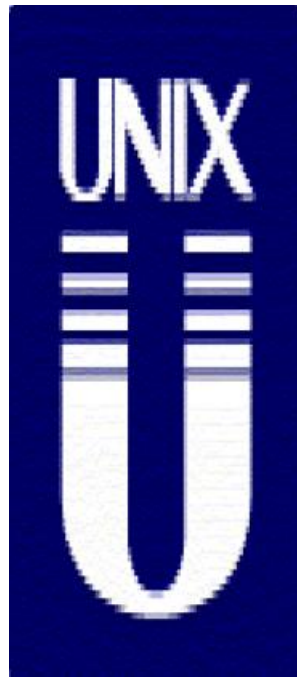
Art of Unix Programming (Cont'd)

- **Rule of Transparency:** Design for visibility to make inspection and debugging easier
- **Rule of Robustness:** Robustness is the child of transparency and simplicity
- **Rule of Least Surprise:** In interface design, always do the least surprising thing
- **Rule of Silence:** When a program has nothing surprising to say, it should say nothing



Art of Unix Programming (Cont'd)

- **Rule of Repair:** When a program must fail, it should show errors immediately and clearly so they can be fixed quickly.
- **Rule of Economy:** Programmer time is expensive; conserve it in preference to machine time
- **Rule of Optimization:** Prototype before polishing. Get it working before you optimize it.
- **Rule of Extensibility:** Design for the future, because it will be here sooner than you think



Symptoms of Rotting Design

Rotting Design means the software design gradually becomes worse over time because of continuous changes, quick fixes, and poor maintenance.

- Rigidity
- Fragility
- Immobility
- Viscosity



Rigidity

A software can be called a rigid when it is difficult to change even in a sample way

- Difficult to change
- Ripple Effect: A small change in one part of the system causes unexpected changes or problems in other parts.
- Entering roach motel: Code is easy to add but hard to remove or improve.

Official Rigidity occurs when managers become afraid of changing software and prevent necessary modifications. A design problem can turn into a management problem, making improvement and maintenance difficult.



Fragility

- ❑ Conceptually irrelevant pieces breaking in multiple places
- ❑ Such software is impossible to maintain. Every fix may create new errors, making the software worse instead of better.
- ❑ Managers and Customers have a feeling that Developers have lost control over the project

Closely related to rigidity is fragility. Fragility means that when we make a change in one part of the software, it can cause problems or errors in many other parts. These problems often happen in areas that are not even connected to the part we changed.



Immobility

- Inability to reuse software
- Software is simply rewritten instead of reused

Immobility means that software parts are difficult to reuse in other projects or even in different parts of the same project. Sometimes a developer needs a module that is similar to one already created by another developer, but the existing module cannot be easily used because it is too dependent on its original design or system.



I am asking for someone NOT everyone

Viscosity

Viscosity means that making changes to software is difficult because the easiest way to do the change is not always the correct way.

- It is easy to do the wrong thing, but difficult to do the right thing.
- When proper design-preserving methods are harder to use than shortcuts or hacks, the viscosity of the design is high.
- Viscosity of the environment occurs when the development tools or processes are slow and inefficient.



S O L I D
S O L I D

SOLID & Beyond....

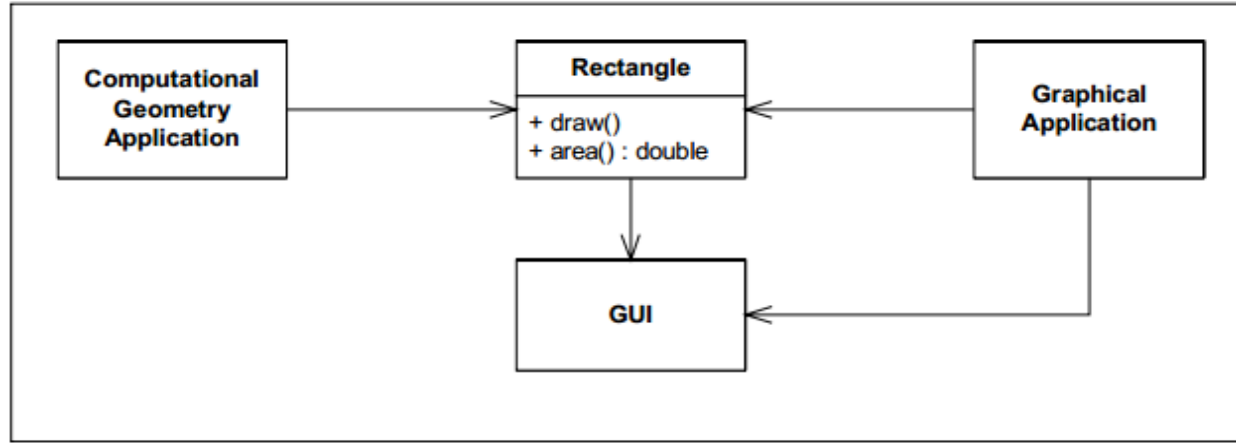
- Single Responsibility Principle (SRP)
- Open Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)
- Tell Don't Ask
- DRY – Don't Repeat yourself
- Once and only once
- The Law of Demeter
- Package Principles
- DRY
- YAGNI

Single Responsibility Principle (SRP)

A class or module should have only one responsibility and should have only one reason to change.



SRP (Cont'd)



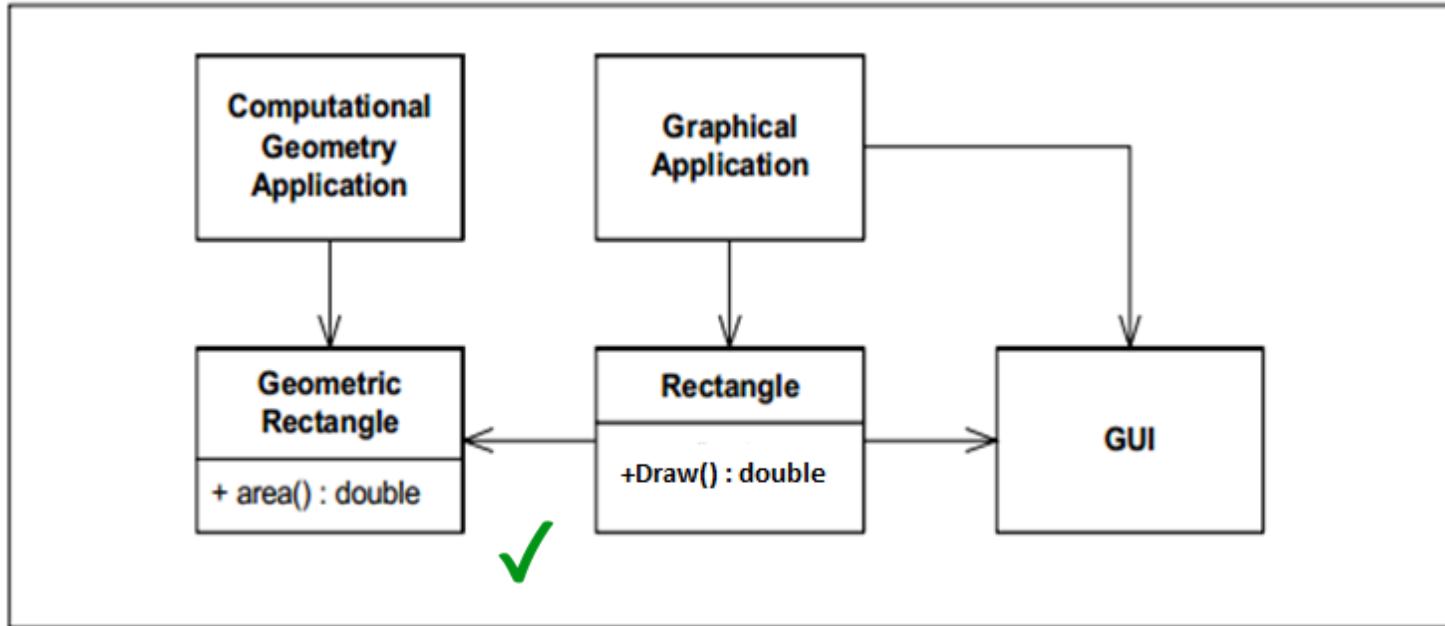
Rectangle class will change for two reasons

The Rectangle class violates the Single Responsibility Principle (SRP) because it has two responsibilities:

- Calculating area for computational applications.
- Drawing the rectangle for graphical applications.

So, the Rectangle class can change for two different reasons, which makes the design harder to maintain.

SRP (Cont'd)



SRP (Cont'd)

```
class User
{
    void CreatePost(Database db, string postMessage)
    {
        try
        {
            db.Add(postMessage);
        }
        catch (Exception ex)
        {
            db.LogError("An error occurred: ", ex.ToString());
            File.WriteAllText(@"\LocalErrors.txt", ex.ToString());
        }
    }
}
```

SRP (Cont'd)

```
class User
```

```
{  
    private ErrorLogger errorLogger = new ErrorLogger();  
    void CreatePost(Database db, string postMessage)  
    {  
        try  
        {  
            db.Add(postMessage);  
        }  
        catch (Exception ex)  
        {  
            errorLogger.Log(ex.ToString());  
        }  
    }  
}
```

```
class ErrorLogger
```

```
{  
    void Log(Database db, string error)  
    {  
        db.LogError("An error occurred: ", error);  
        File.WriteAllText(@"LocalErrors.txt", error);  
    }  
}
```

Practice 17: Applying SRP

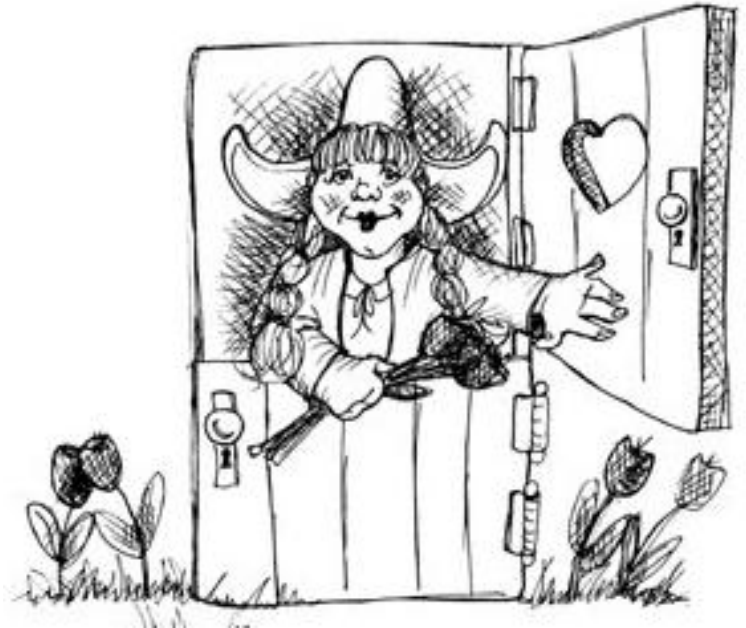
In the provided code, the OnDemandAgentService class violates the Single Responsibility Principle (SRP) by handling **multiple responsibilities**.

[Code link](#)

By refactoring, please ensure that each class is focused on a single responsibility, making the system more maintainable, testable, and scalable.

Open Closed Principle (OCP)

A module should be
open for extension but
closed for modification



OCP (Cont'd)

OCP violation

```
while (rentals.hasMoreElements()) {  
    double thisAmount = 0;  
    Rental each = (Rental)rentals.nextElement();  
  
    //determine amounts for each line  
    switch (each.getMovie().getPriceCode()) {  
        case Movie.REGULAR:  
            thisAmount += 2;  
            if (each.getDaysRented() > 2)  
                thisAmount += (each.getDaysRented() - 2) * 1.5;  
            break;  
        case Movie.NEW_RELEASE:  
            thisAmount += each.getDaysRented() * 3;  
            break;  
        case Movie.CHILDRENS:  
            thisAmount += 1.5;  
            if (each.getDaysRented() > 3)  
                thisAmount += (each.getDaysRented() - 3) * 1.5;  
            break;  
    }  
}
```

OCP (Cont'd)

How this diagram follows OCP:

Before (bad design):

The Movie class directly calculates charges.

If a new price type is added (for example, "Premium Price"), we must modify the Movie class.

This violates OCP because existing code needs to change.

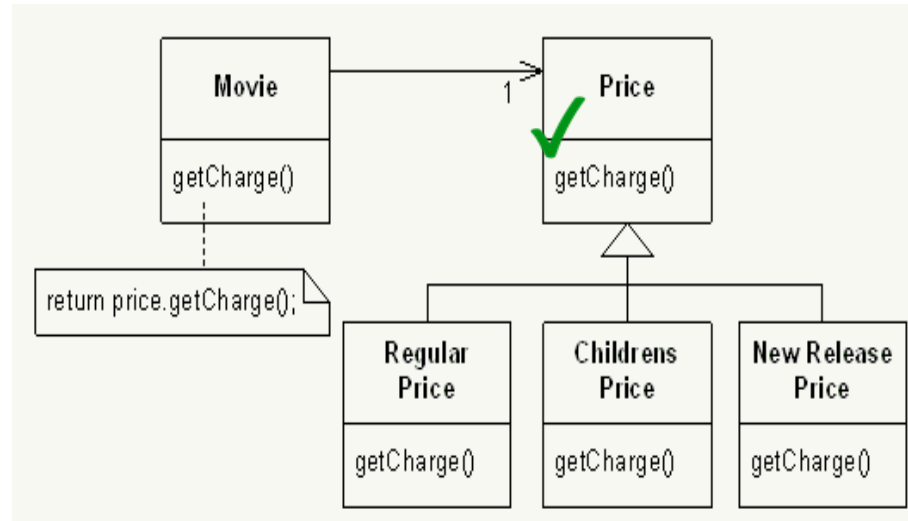
After (shown in the diagram):

Movie depends on the Price class (abstraction).

Regular Price, Children's Price, and New Release Price are subclasses of Price.

Each price type has its own `getCharge()` method.

Now, if we need a new pricing type we simply create a new subclass of Price.



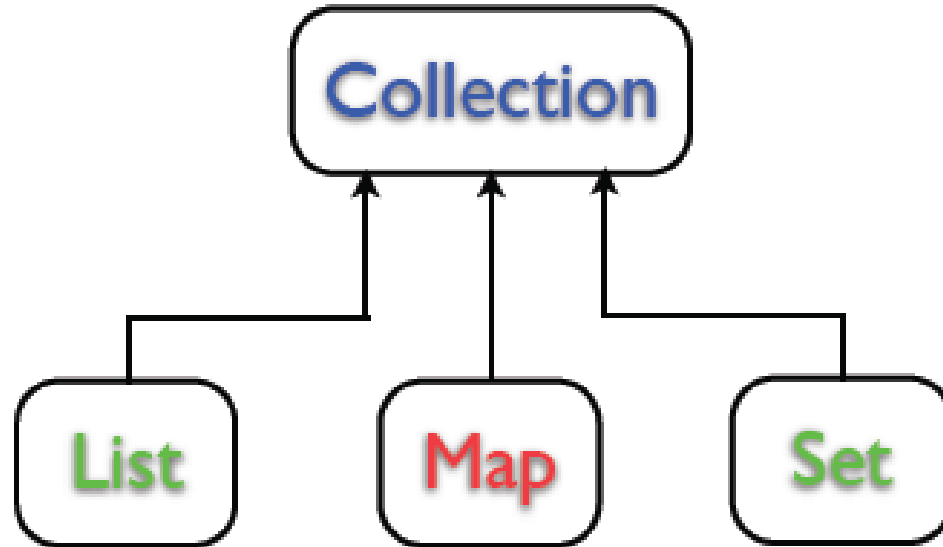
Liskov Substitution Principle (LSP)

Liskov Substitution Principle (LSP) means that a subclass should be able to replace its parent class without causing any problems.



LSP Violation

LSP (Cont'd)

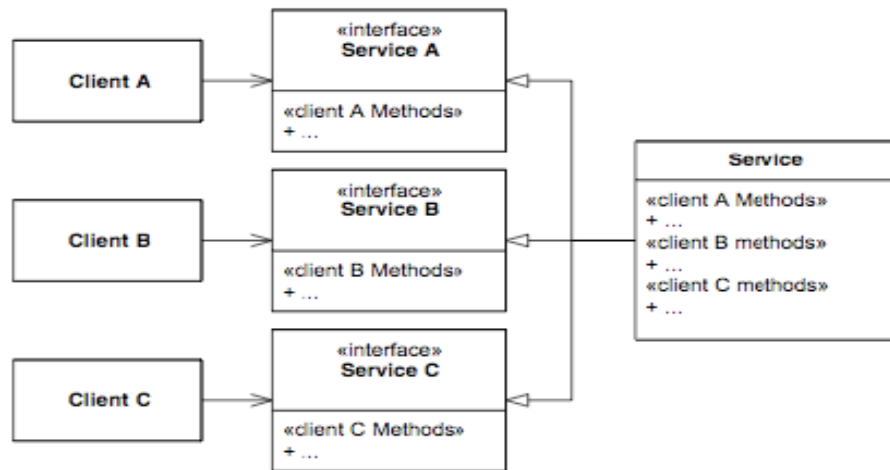
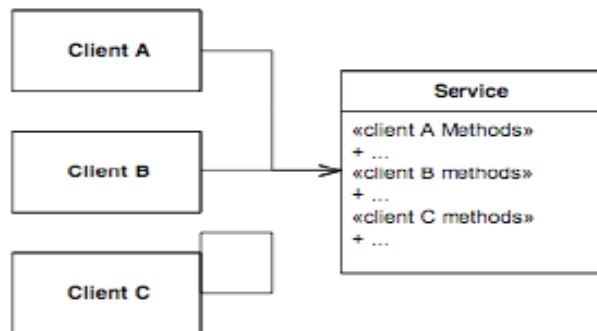


Interface Segregation Principle (ISP)

Interface Segregation Principle (ISP) means that it is better to have small, specific interfaces for different clients instead of one large general-purpose interface.

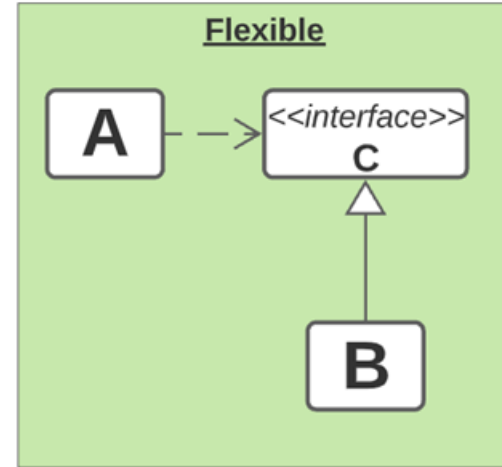
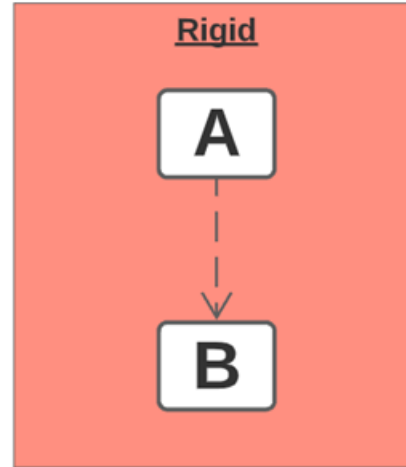


ISP (Cont'd)

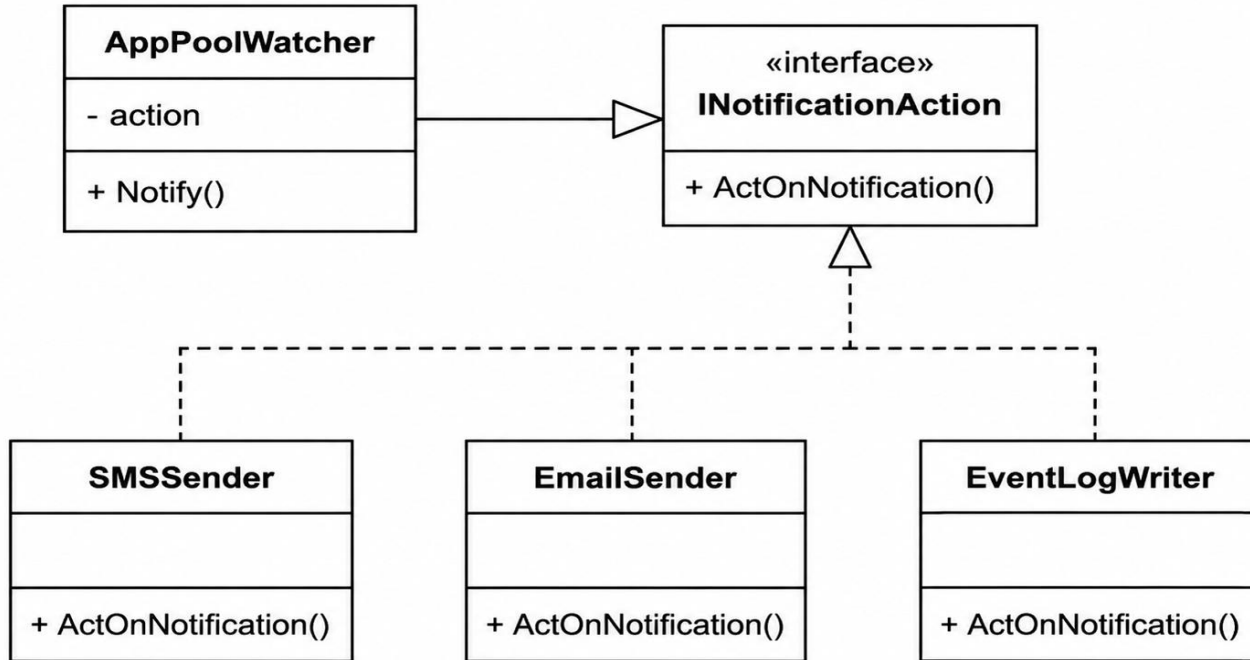


Dependency Inversion Principle (DIP)

Dependency Inversion Principle (DIP) means that high-level modules should depend on abstractions (interfaces), not on specific implementations (concrete classes).



DIP (Cont'd)



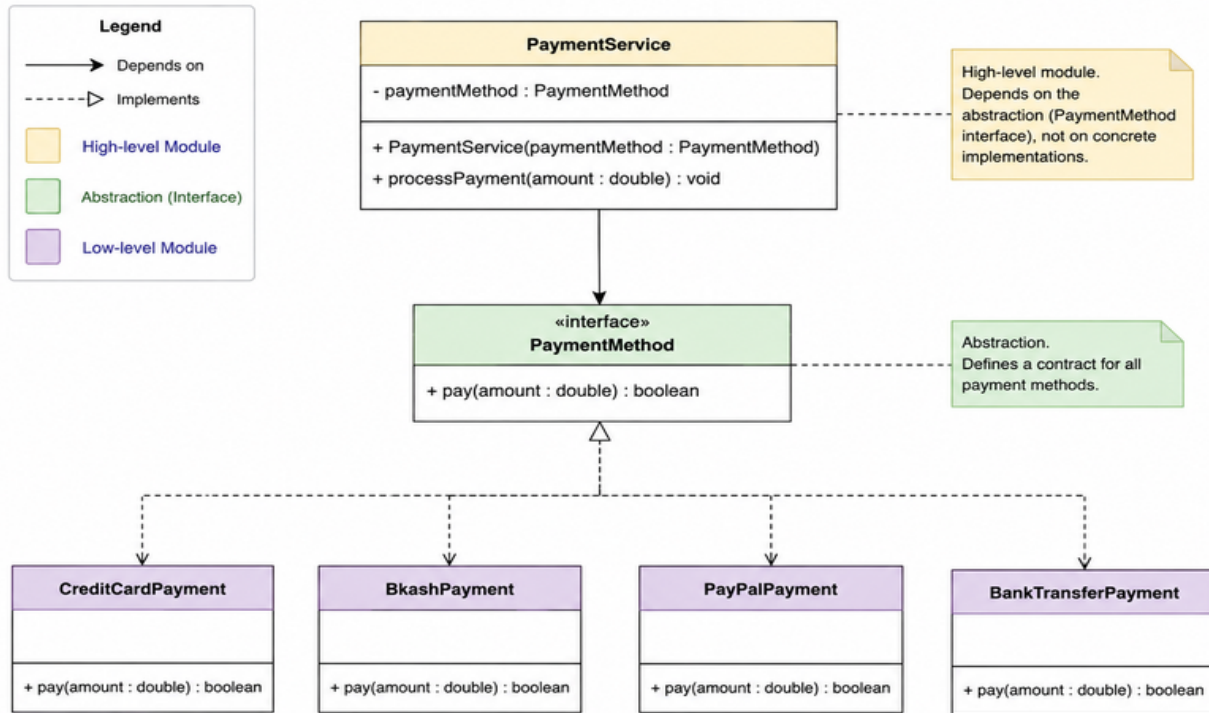
Practice 18: Applying DIP

Payment Processing System

In today's digital commerce, customers use various payment methods to complete transactions. A well-designed payment processing system should handle different payment methods such as Credit Card, bKash, PayPal, and Bank Transfer, ensuring scalability to integrate future payment methods seamlessly.

Your task is to design a system that adheres to the Dependency Inversion Principle (DIP), allowing the high-level PaymentService module to remain independent of specific payment method implementations.

Practice 18: Applying DIP



How it works:

1. **PaymentService** (high-level module) depends on **PaymentMethod** abstraction.
2. Concrete payment classes implement the **PaymentMethod** interface.
3. New payment methods can be added by creating new classes that implement **PaymentMethod**.
4. No changes are required in **PaymentService** when adding a new payment method.

Tell Don't Ask



Tell Don't Ask (Cont'd)

```
if (getStateOfSomething())  
    doThis();  
else  
    doThat();
```

v/s

```
object.doSomethingForMe();
```

An object should tell another object what to do instead of asking for its internal data and making decisions itself.

DRY – Don't Repeat yourself

The DRY principle states that every piece of knowledge in a software system should have a single, clear, and authoritative representation.

Practice -19: Download this code and refactor it using DRY principle

Refactoring has as a primary objective getting each piece of functionality to exist in exactly one place in the software.

–Ron Jeffries

Once and only once

The Law of Demeter

Any method of an object should only call methods belonging to:

- itself
- Immediate friends
- any parameters that were passed in to the method
- any objects it created

Train Wreck

- A train wreck refers to a long chain of method calls between objects, which makes code difficult to understand and maintain. An object should only communicate with its closely related objects and should not directly access methods of objects returned by other objects.

```
my_television.front_panel.switches.power.on()  
;
```

v/s

```
my_television.power_up();
```

- Talk only to your neighbors, not with their neighbors.



Train Wreck

What is the problem here?

- **Deep Chaining:** `order.Customer.Address.Street` and `order.Customer.Address.City` traverse multiple levels of objects, forming a "train wreck."
 - Reduced readability
 - Increased Fragility
 - Makes Testing Harder
- **Tight Coupling:** The `OrderProcessor` class is tightly coupled to the internal structure of `Order`, `Customer`, and `Address`. Any change to these structures will ripple through the code.
- **Encapsulation Violation:** The `Order` and `Customer` classes expose too much of their internal details, reducing encapsulation.

YAGNI (You ain't gonna need it)

What If.....

"Do not implement functionality until it is actually required."

Developers should avoid adding extra features, classes, or complexity based on future assumptions. Only build what is needed for the current requirements.



YOU AIN'T GONNA NEED IT

Don't waste resources on what you *might* need.

YAGNI (You ain't gonna need it)

```
public class User
{
    public string Name { get; set; }
    public string Email { get; set; }

    // Adding future functionality that is not currently needed
    public List<string> Permissions { get; set; } = new List<string>();
    public List<string> Roles { get; set; } = new List<string>();

    public void GrantPermission(string permission)
    {
        Permissions.Add(permission); // Not currently used
    }

    public void AssignRole(string role)
    {
        Roles.Add(role); // Not currently used
    }

    public string GetUserInfo()
    {
        return $"{Name} ({Email})";
    }
}
```

```
public class User
{
    public string Name { get; set; }
    public string Email { get; set; }

    public string GetUserInfo()
    {
        return $"{Name} ({Email})";
    }
}
```

YAGNI (You ain't gonna need it)

YAGNI is a software development principle that means do not add functionality until it is actually needed.

Why YAGNI...

- Cost of Building - Wrong features
- Cost of Delay - If you work for the next Sprints, you will fail this Sprint
- Cost of Carry - Build wrong and carry it
- Cost of Repair - Build wrong and repair overtime

KISS (Keep It Simple, Stupid)

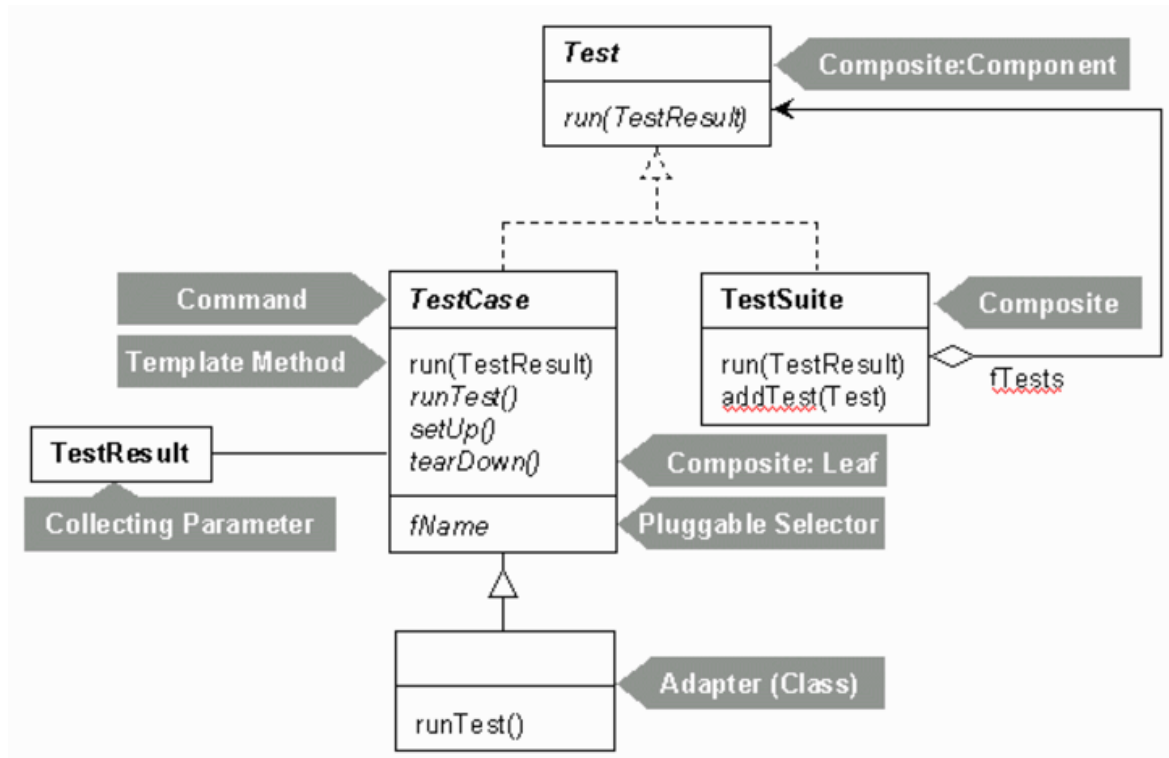
KISS (Keep It Simple, Stupid) is a software design principle that encourages developers to keep code and designs simple by avoiding unnecessary complexity. It helps create software that is easier to understand, maintain, test, and modify.

“Any intelligent fool can make things bigger, more complex, and more violent. It takes a touch of genius—and a lot of courage to move in the opposite direction”

- E. F. Schumacher



KISS (Keep It Simple, Stupid)



The JUnit framework, designed by Kent Beck and Erich Gamma, is a great example of modular system design, promotes simplicity, extensibility, and adherence to object-oriented principles. [Find details here](#)